LOSSLESS DATA COMPRESSION TOOL

TAI PROJECT #1 - 2025/26

Guilherme Rosa
Student ID: 113968
Universidade de Aveiro
Aveiro, Portugal
guilherme.rosa@ua.pt

João Roldão
Student ID: 113920
Universidade de Aveiro
Aveiro, Portugal
joao.roldao@ua.pt

Henrique Teixeira
Student ID: 114588
Universidade de Aveiro
Aveiro, Portugal
henrique.teixeira@ua.pt

April 11, 2026

ABSTRACT

We developed and benchmarked a lossless compression tool for the TAI course project, centered on statistical coding methods based on range-coder-style entropy coding. The repository now exposes three distinct design points. `g07-v9` is the strongest compression-oriented project variant in the current benchmark snapshot, reaching a 54.40% average compressed size on the 8-file corpus (13 MiB total). `g07-v8` is the fastest G07 variant in average compression and decompression time, using an LZ77-style token stream without entropy coding and averaging 26 ms compression and 24 ms decompression. Even so, the most balanced overall design is `g07-v5`: it combines BWT, MTF, optional zero-run encoding, and an adaptive order-1 model with range coding/rANS backends, achieving 54.77% while remaining far faster to compress than `g07-v9` and much more compact than `g07-v8`. The resulting comparison highlights the main project trade-off between compression strength, throughput, and overall practical balance.

Keywords Data Compression · Burrows-Wheeler Transform · PPM · Range Coding · rANS · Parallel Processing

1 Introduction

Lossless compression is fundamentally a two-part problem: the encoder must first model the input so that probable symbols receive high estimated probability, and only then can the coder map those probabilities into short bit representations. In practice, the quality of a compressor depends on how well the model captures structure in the data and how efficiently the coding stage turns that structure into a compact stream.

This project was developed in the context of the Algorithmic Information Theory course and evolved through several compression strategies. The repository now contains three especially relevant variants. `g07-v5` is the clearest balanced milestone and still offers the best overall speed/ratio balance; `g07-v9` extends the same statistical family and achieves the best compression ratio among the project versions; and `g07-v8` explores the opposite design point, replacing the statistical pipeline with a very fast LZ77-style matcher.

The goal of this report is therefore not to narrate every version in detail, but to explain the main design choices, summarize the implementation strategy, and present the measured trade-offs on the benchmark corpus. In particular, we use `g07-v5` as the main case study for the project's best overall compromise, `g07-v9` as the ratio leader in the current repository snapshot, and `g07-v8` as the speed-oriented contrast.

2 Background

2.1 Information Theory

Shannon's source coding view remains the right abstraction for this project: compression is only possible when the source is not uniformly random and the model can exploit that deviation from randomness [1]. Entropy estimates were therefore used in the repository as a simple decision signal for model selection and for falling back to uncompressed output on near-random files.

2.2 Transform Coding

The Burrows–Wheeler Transform (BWT) rearranges a block so that symbols occurring in similar contexts become clustered [2]. On its own, BWT is reversible but not compressive; its value comes from making the transformed stream easier to encode. The usual follow-up is Move-to-Front (MTF), which converts these clusters into many small ranks, often producing long runs of zeros. A lightweight run-length stage can then exploit those zeros before entropy coding.

2.3 Statistical Modeling

For the modeling stage, the repository explores both order-0 and order-1 byte models. Order-0 uses only global symbol frequencies, while order-1 conditions the next byte on the previous byte and is therefore closer to a practical Prediction by Partial Matching (PPM) scheme [3]. The order-1 path must also define an escape mechanism for unseen symbols so that the encoder and decoder can safely fall back to a lower-order distribution.

2.4 Entropy Coding

The main statistical line of development uses range coding as the core entropy coder [4, 5]. The repository also experiments with rANS as a faster static alternative [6]. In contrast, the late `g07-v8` experiment deliberately removes entropy coding and uses an LZ77-style token stream instead [7, 8]; this makes it useful as a speed comparison, but not as the main statistical design.

3 Methodology

3.1 System Architecture

The report uses a version-focused methodology. Rather than treating the repository as a single static compressor, we identify three representative designs. `g07-v5` is the main balanced design, combining classic modeling/coding separation with practical runtime. `g07-v9` is the stronger compression-oriented extension of the same family. `g07-v8` is the speed-focused contrast that removes the statistical pipeline and uses direct dictionary matching. This framing exposes the central trade-off between ratio, throughput, and overall balance.

3.2 Algorithm Selection

The `g07-v5` line remains the primary subject because it offers the best overall speed/ratio trade-off in the refreshed benchmark set and best illustrates the repository's statistical compression pipeline. BWT, MTF, and zero-run coding reshape low-entropy structure before modeling; order-1 adaptive modeling captures local byte dependencies better than a static order-0 model; and range coding/rANS provide the coding stage while keeping the implementation efficient. `g07-v9` is included because it improves ratio further through per-block candidate selection and broader transform choices, while `g07-v8` is kept as a comparison point because its strength is speed, not compactness.

3.3 Optimization Strategy

Development proceeded iteratively. Earlier versions established the basic order-0 and range-coding path, then introduced BWT preprocessing and faster data structures. The `g07-v5` line consolidated the strongest ideas into one pipeline: block-based BWT, MTF, optional zero-run encoding, adaptive order-1 modeling, and parallel block processing. `g07-v8` then explored the opposite extreme by discarding preprocessing and entropy coding entirely in order to maximize speed. Finally, `g07-v9` extended the statistical line with per-block candidate search over multiple transforms and model orders to push ratio further.

3.4 Benchmarking

All claims in this report are based on the refreshed benchmark artifacts already present in the repository. The benchmark corpus contains eight files (A–H) totaling 13 MiB, with different characteristics including text, structured binaries, high-entropy data, and an incompressible random case. The reported metrics are compressed size, compression ratio, bits per symbol, compression time, decompression time, and lossless correctness. The repository benchmarks compare `g07-v5`, `g07-v8`, and `g07-v9` against gzip, bzip2, xz, and zstd; in this report, aggregate numerical comparisons are taken from `benchmarks/results.md` regenerated on 2026-04-11, and file-level examples are taken from the same benchmark set.

4 Implementation

4.1 BWT-Based Statistical Line

The main statistical line in the repository is block-oriented. In `g07-v5`, structured data is passed through BWT so that symbols occurring in similar contexts are grouped together. This creates the distributional skew later exploited by MTF and entropy coding. The implementation history shows that larger and better-balanced blocks improved compression because they preserved more context across each transform pass.

4.2 Move-to-Front and Run-Length Encoding

After BWT, the transformed block is passed through MTF. This turns repeated local symbol clusters into many low ranks, especially zeros. A zero-run stage is then optionally applied to shrink long zero runs further. The value of this preprocessing chain is practical rather than theoretical: it reshapes the byte stream into a form that a simple context model can code effectively. The repository notes one important limitation here, namely the handling of rank 255 in the zero-run stage, which helps explain why some binary-heavy files remain difficult.

4.3 Context Models and Coders

The statistical core of `g07-v5` is an adaptive order-1 model implemented over flat arrays instead of pointer-heavy dynamic structures. This keeps the model compact and cache-friendly. The design also incorporates a PPM-style escape path: when the current order-1 context cannot emit a symbol directly, the coder escapes to a lower-order distribution. Two refinements described in the repository are especially relevant: Method C exclusions, which remove already-seen symbols from the fallback distribution, and singleton-based escape estimation, which makes escape probabilities depend on how many symbols are still weakly supported in the current context.

The compliant coding stage is based on range coding, with rANS used in some static order-0 cases as a faster alternative backend. In architectural terms, `g07-v5` remains a modeling-first compressor: preprocessing and prediction define a probability model, and the coder converts that model into the final bitstream.

4.4 From v5 to v9

The main architectural change in `g07-v9` is that it no longer commits to one preprocessing path for the entire file. Instead, it performs parallel per-block candidate search over raw, BWT-based, LZP-prefiltered, and x86-filtered variants, and it can choose between order-1 and order-2 statistical models. The selected block configuration is stored with explicit parallel-header metadata and transform flags, allowing the decompressor to replay the exact sequence of transforms. This broader search explains why `g07-v9` improves ratio over `g07-v5`, but it also explains the much larger compression cost.

4.5 Speed-Oriented Variant

By contrast, `g07-v8` follows a much simpler implementation strategy. It uses a single-pass LZ77-style matcher with a small hash table, byte-aligned tokens, and no entropy coder. This makes the code much smaller and much faster, but it also removes the stronger statistical modeling that makes `g07-v5` and `g07-v9` competitive on compression ratio.

5 Results

5.1 Compression Ratio Evolution

The repository history shows a clear progression from a simple order-0 baseline toward stronger BWT-based statistical compressors. The key milestones for the purposes of this report are `g07-v5`, which consolidated the best balanced requirement-compliant implementation; `g07-v8`, which optimized for speed rather than ratio; and `g07-v9`, which pushed the statistical line further and became the strongest G07 variant in compression ratio.

5.2 Benchmark Comparison

Table 1: Aggregate benchmark results from the 8-file repository corpus. Lower ratio is better.

Tool	Avg. ratio	Avg. comp. time	Avg. decomp. time
g07-v5	54.77%	98 ms	101 ms
g07-v8	76.42%	**26 ms**	**24 ms**
g07-v9	**54.40%**	789 ms	126 ms
gzip	59.49%	111 ms	22 ms
bzip2	54.88%	177 ms	90 ms
xz	54.16%	456 ms	65 ms
zstd	58.58%	20 ms	13 ms

Table 1 shows three distinct optima. `g07-v9` is the strongest project version in compression ratio at 54.40%, improving on `g07-v5`'s 54.77% and slightly beating bzip2's 54.88%, although it still remains behind xz at 54.16%. `g07-v8` is the clear speed winner among the project versions. Even so, `g07-v5` remains the most balanced overall design: its ratio stays close to `g07-v9`, while its compression time is far lower and its output quality is vastly better than `g07-v8`.

The file-level results show a more nuanced picture. `g07-v9` wins files A, E, and G; `g07-v5` wins B and D; xz wins C and H; and bzip2 wins F. Against bzip2 specifically, `g07-v9` produces smaller outputs on A, B, C, D, E, and G, but loses on F and H. The gains are most convincing on structured inputs such as A and G, where the broader per-block search of `g07-v9` can choose a better transform/model combination than the fixed `g07-v5` pipeline.

The contrast with `g07-v8` is even sharper. On file B, the ratio is 17.35% for both `g07-v5` and `g07-v9`, but 37.84% for `g07-v8`; on file G, the values are 28.82%, 28.48%, and 72.76%; and on file H they are 43.79%, 43.64%, and 58.02%. In other words, `g07-v8` demonstrates that much of the runtime cost in the statistical variants comes from modeling and coding sophistication rather than from file I/O or framework overhead.

5.3 Performance Analysis

From a systems perspective, the repository exposes three practical conclusions. First, stronger modeling is worthwhile on structured inputs because the ratio improvements are real, but the extra search cost can become large. Second, parallel block processing is important for making a BWT-based pipeline practical. Third, once entropy coding and preprocessing are removed, throughput increases dramatically, but the output size deteriorates so much that the result is better seen as a different product goal rather than a direct replacement. This is why `g07-v5` still stands out as the best overall balance, even though `g07-v9` compresses slightly better and `g07-v8` runs much faster.

5.4 Ablation Studies

The version history also acts as an ablation study. Earlier versions without BWT or without the later order-1 refinements are consistently worse in ratio, while the v5 documentation attributes its gains to a combination of Method C exclusions, improved escape estimation, adaptive block sizing, and block-level parallelism. `g07-v9` then shows that further gains come from widening the search space rather than from one isolated trick. No single optimization explains the final result: the competitive ratios come from several modest improvements that compound well.

6 Conclusion

6.1 Summary of Achievements

The repository demonstrates a credible lossless compression project built around the modeling/coding principles emphasized in the course. In the refreshed benchmark set, `g07-v9` is the strongest G07 variant in compression ratio at

54.40%, while `g07-v8` is the fastest project variant. Even so, `g07-v5` remains the best overall balance: it combines BWT-based preprocessing, adaptive order-1 modeling, and entropy coding into a compressor that stays close to `g07-v9` in ratio while avoiding the extreme search cost of that version.

6.2 Lessons Learned

Three lessons stand out. First, preprocessing and modeling must be designed together: BWT and MTF were valuable not by themselves, but because they made the downstream model easier to code efficiently. Second, many of the final gains came from engineering details such as better data structures, escape handling, block scheduling, and broader candidate selection rather than from one dramatic algorithmic change. Third, the late `g07-v8` experiment shows that speed and ratio can diverge sharply; removing entropy coding produces an excellent throughput-oriented compressor, but not one that serves the same objective.

6.3 Limitations

The main limitations are visible on difficult binary-heavy files, where even the stronger statistical variants still lose to specialized baselines such as xz or bzip2 on particular corpus members. The repository also shows that pushing ratio further, as in `g07-v9`, can incur a very large runtime cost, which reinforces the need to distinguish between best compression and best overall practical balance.

6.4 Future Work

Future work could therefore follow two directions. One direction is to keep the `g07-v5`/`g07-v9` statistical philosophy and reduce the cost of broader candidate search while improving difficult cases through better transforms or stronger adaptive models. The other is to continue the `g07-v8` direction as a separate high-throughput compressor for scenarios where speed matters more than compactness. The most important result of the project is that the repository now contains evidence for all three trade-off points, supported by a common benchmark methodology.

References

[1] Claude E Shannon. A mathematical theory of communication. *The Bell system technical journal*, 27(3):379–423, 1948.

[2] Michael Burrows and David J Wheeler. A block-sorting lossless data compression algorithm. In *Digital SRC Research Report*, 1994.

[3] John Cleary and Ian Witten. Data compression using adaptive coding and partial string matching. *IEEE transactions on Communications*, 32(4):396–402, 1984.

[4] Jorma Rissanen and Glen G Langdon. Arithmetic coding. *IBM Journal of research and development*, 23(2):149–162, 1979.

[5] Michael Schindler. A fast renormalisation for arithmetic coding. Technical report, 1998.

[6] Jarek Duda. Asymmetric numeral systems. *arXiv preprint arXiv:0902.0271*, 2009.

[7] Jacob Ziv and Abraham Lempel. A universal algorithm for sequential data compression. *IEEE Transactions on Information Theory*, 23(3):337–343, 1977.

[8] Yann Collet. Lz4: Extremely fast lossless compression algorithm. `https://lz4.org/`. Accessed 2026-04-11.